CS 61A — Week 1

Solutions: Discussion 0, Lab 0, HW 1, past-exam excerpt

Lab 0 Q1–Q2. **Idea.** An expression computes a value; a return statement makes that value the call's result.

1. The numeral `2000` evaluates to 2000; `2000 + 26` evaluates to 2026. At the prompt, each result is displayed.
2. Since $45^2 = 2025$, a permitted return expression is `45**2 + 1`. It uses only numbers and allowed arithmetic.

```
def twenty_twenty_six():
    return 45**2 + 1
```

Discussion 0 Q1–Q2. **Idea.** Apply the worksheet's ordering and group-membership rules; these activities have no single numeric answer.

1. In the Name Game, each new participant lists earlier participants in reverse turn order. With Eva, Lem, Alyssa, Alyssa names herself, then Lem, then Eva.
2. In Partitions, a candidate fact must hold for *every* person in one half and *no* person in the other. Check both conditions. A visible characteristic or preference is ineligible even if it separates the groups.
3. Repeat with a different equal split, then test the final whole-group fact with the TA. The answers depend on actual participants.

Discussion 0 Q3. **Idea.** Work backward through f and g until a decimal split can be built cheaply from 5.

1. $2026 = f(2025)$ and $2025 = g(1013)$. The digits of 1013 split as 10 and 13, distinct positive integers.
2. $10 = f(g(5))$ takes two calls. $13 = g(f(f(5)))$ takes three. Joining and applying the two outer calls takes another three.
3. Thus `f(g(h(f(g(5)),g(f(f(5))))))` is valid and has eight calls. The computed intermediate values are $9, 10$ and $6, 7, 13$, then $1013, 2025, 2026$.
4. To check fewest calls, construct all attainable values at each call count. For n total calls, the last call is either unary with $n-1$ calls inside, or binary with operand counts k and $n-1-k$. All intermediate values are at least 5, and each operation strictly increases its inputs, so values above 2026 can be excluded. The exhaustive check finds no 2026 with at most seven calls.

1. The expression has 27 characters without spaces, including its comma. The formula $3n + 2b + 1$ shows that nine or more calls require at least 28 characters.
2. With eight calls and no binary calls, $g(x) > f(x)$ for every attainable $x \geq 5$. Both functions are increasing, so eight successive g calls maximize the result, giving 1025. Thus reaching 2026 needs at least one binary call and at least 27 characters. Our expression attains that bound.

HW 1 Q1: A Plus Abs B. **Idea.** Select the operation by the sign of b, keeping the supplied final call unchanged.

1. If $b < 0$, $|b| = -b$, so $a + |b| = a - b$: bind `f` to `sub`.
2. Otherwise $|b| = b$, so bind `f` to `add`. The zero case belongs here and returns a.
3. The common final call now computes the desired value in either case; only the two allowed blanks were filled.

```
from operator import add, sub

def a_plus_abs_b(a, b):
    if b < 0:
        f = sub
    else:
        f = add
    return f(a, b)
```

For $(2, 3)$ and $(2, -3)$ the result is 5; for $(-1, 4)$ and $(-1, -4)$ it is 3.

HW 1 Q2: Two of Three. **Idea.** Compare the three possible pairs. Squaring preserves the order of positive inputs, so the smallest pair has the smallest square sum.

1. Write the ordered inputs as $a \le b \le c$. Positivity gives $a^2 \le b^2 \le c^2$, so $a^2 + b^2 \le a^2 + c^2$ and $a^2 + b^2 \le b^2 + c^2$. The minimum of the three pair sums is the desired one.
2. Ties are allowed: if all inputs are 5, each pair sum is 50. Taking their minimum still returns 50.

```
def two_of_three(i, j, k):
    return min(i*i + j*j, i*i + k*k, j*j + k*k)
```

The supplied tests give $1 + 4 = 5$, $9 + 1 = 10$, $4 + 64 = 68$, and $25 + 25 = 50$. With exact integers, summing all three squares and subtracting one largest square also works. For floating-point inputs this subtraction can lose the smaller squares: with $(1.0, 2.0, 10^{20})$ it returns 0.0 instead of 5.0. The pairwise method avoids that cancellation.

HW 1 Q3: Largest Factor. **Idea.** Find the smallest divisor at least 2; the factor paired with it is the largest proper factor.

```
def largest_factor(n):
    d = 2
    while n % d != 0:
        d = d + 1
    return n // d
```

1. Before each test, no integer from 2 through $d - 1$ divides n. A failed test extends this fact when d increases by 1.
2. The loop terminates by $d = n$, because $n \% n = 0$. It stops at the least divisor $d \ge 2$.
3. The output n/d divides n and is smaller than n. Any other proper divisor q has $n/q \ge d$, so $q \le n/d$. The output is therefore largest.

For 15, tests $d = 2, 3$ yield 5; for 80, $d = 2$ yields 40; for prime 13, $d = 13$ yields 1. A valid alternative tests $n - 1, n - 2, \ldots, 1$ and returns the first divisor.

HW 1 Q4: Hailstone. **Idea.** Print and count the initial term before replacing anything; then each iteration produces exactly one additional term.

```
def hailstone(n):
    print(n)
    length = 1
    while n != 1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
        print(n)
        length = length + 1
    return length
```

1. Before each loop test, `n` is the last printed term, and `length` equals the number of terms printed so far.
2. If `n != 1`, the parity test computes exactly the next term. Printing it and increasing `length` preserves that statement.
3. If the sequence reaches 1, the test becomes false after 1 has already been printed. Return the count. For input 1, this happens immediately and returns 1.

Input 10 prints $10, 5, 16, 8, 4, 2, 1$, one per line, and returns 7. Input 27 prints 112 terms, with maximum 9232, and returns 112. These executions do not establish termination for every positive integer.

Fall 2025 Midterm 1, Q1(a), p. 3 (excerpt). **Idea.** The second assignment uses the old `about`; nested printing then receives a `None` argument.

1. First `about = -1`. The next right-hand values are 6 and $(-1) \cdot 7 = -7$; afterwards `about = 6`, `face = -7`.
2. Inner `print(face)` displays `-7` and returns `None`. Outer `print(None)` displays `None` and also returns `None`.

Displayed output is exactly

```
-7
None
```

The final returned value is `None`; it adds no automatic prompt output.